

TTFL: Trust-Flow-Driven Federated Learning with TEE-Anchored Admission

Anonymous Author(s)

Abstract. Federated learning (FL) enables collaborative edge intelligence without moving raw data, but open participation couples two sources of uncertainty: the server must assess whether a client executed an admissible training process, and it must determine whether the resulting update is useful under Non-IID data or carries a poisoning objective. A static TEE admission gate cannot resolve the second question, whereas a single update-quality threshold can remove legitimate long-tail clients. This paper presents TTFL, a trust-flow framework that keeps execution credibility, current update content, long-term utility, and short-term risk as distinct evidence streams until aggregation. A TEE-anchored trusted management and attestation agent (TMAA) supplies admission evidence; an explicit Kalman update and a dual-stream Beta/EMA state model provide cold-start and cross-round memory; and a layer-wise rule performs risk-dependent clipping and data-size-aware survivor re-normalization. The evaluation uses two real Intel/AMD TEE hardware configurations with Flower/PyTorch and CIFAR-10, five seeds per platform (ten platform-seed runs), 20 admitted clients, six attack behaviors, and a 30% malicious setting. Targeted ASR is macro-averaged over the three targeted behaviors, while untargeted attacks are evaluated with clean accuracy, loss, and state-transition metrics. TTFL reaches 92.31% clean accuracy, 10.21% targeted ASR under the stated protocol, and no permanent false ban in the tested runs.

Keywords: Federated Learning · Edge Computing · Trusted Execution Environment · Risk Modeling · Layered Aggregation · Backdoor Defense

1 Introduction

FL trains a global model through parameter exchange while keeping data on edge devices [1,2]. In open edge networks, the server must distinguish malicious updates from legitimate deviations caused by skewed data, while also checking client identity and execution integrity. Compound attacks make this distinction harder by combining poisoning, targeted backdoors, and cross-round camouflage. These requirements are coupled at the decision point: execution evidence constrains which reports can be trusted, update evidence describes the current contribution, and temporal evidence determines whether an isolated anomaly is persistent. Treating them as one score either hides the source of a decision or makes the threshold overly sensitive to Non-IID drift.

Geometric, statistical, reference-based, and temporal defenses address update anomalies [3,4,5,6,8], while backdoor defenses inspect model behavior, clusters, or collusion [9,10,12,13]. These filters cannot prove how local training was executed and can mistake Non-IID drift for attacks. TEE-assisted FL supplies an execution root of trust [15,16,18,19], but static admission does not establish semantic update safety.

This paper proposes *Trust Flow*, a system-level decision framework for trusted aggregation in edge FL. Its contribution is the explicit timing and interface between established mechanisms, rather than a new cryptographic primitive or a new Byzantine estimator: execution credibility is admitted first, update content is audited next, temporal evidence is updated afterward, and only then are layer privileges assigned. TMAA provides execution-side evidence, while the server-side trust flow converts that evidence and update probes into a cross-round state. The contributions are:

1. A precisely initialized admission state that combines TEE-style attestation, runtime entropy, and a covariance-aware Kalman recursion, including a conservative first-round policy for clients without history.
2. A dual-stream state model with explicit Beta updates for historical utility and an EMA of the strongest current risk probe, so benign distributional drift and attack impulses are not forced into one score.
3. A layer-wise aggregation rule that retains the original data-size weight, applies risk-dependent thresholds and clipping, and re-normalizes only the surviving mass.
4. A paired-platform evaluation under controlled Non-IID CIFAR-10 attacks, together with ablations and an explicitly out-of-assumption boundary stress test.

2 Background and Related Work

Federated learning in edge computing. Edge deployment creates a trust-continuity problem: a valid client can submit a semantically unsafe update, while a benign long-tail client can appear anomalous. The decision must therefore preserve execution and update evidence separately before aggregation.

Poisoning and backdoor attacks. Sign flipping and scaling disrupt convergence, whereas trigger, clean-label, and semantic backdoors implant targeted errors while preserving clean behavior. Such updates can remain directionally plausible [20,21], and attackers can accumulate trust before intermittent injection.

Active defenses and their limits. Krum and Trimmed Mean suppress geometric outliers [3,4]; FLTrust uses a clean direction [5]; and FoolsGold and FLDetector exploit cross-client or cross-round evidence [6,8]. More recent defenses inspect model representations, backdoor behavior, collusion, or privacy-preserving updates, including DeepSight, FLAME, CrowdGuard, RoseAgg, FLPurifier, ShieldFL, and RaSA [9,10,12,13,11,14,7]. TMT-FL and

related TEE-assisted systems further connect execution integrity with model training [19]. These methods address different evidence spaces and therefore are not interchangeable baselines without a common attack protocol and implementation interface. Trust Flow integrates the evidence flow while retaining the distinction between current content and cross-round state; the reported table uses matched classical implementations, and the recent methods are discussed as positioning references rather than assigned unverified numerical results.

TEE-assisted trusted training. TEEs isolate execution and key material, supporting FL integrity and attack mitigation [15,16,17,18,19]. A quote authenticates a measured path and freshness, but not labels or semantic intent; TTFL therefore uses attestation as admission evidence rather than proof of a benign update.

3 System Model and Threat Assumptions

The system contains a central server $\mathcal{S}$ and K edge clients $\mathcal{C} = \{c_1, \dots, c_K\}$. In round t , admitted clients receive $W^{(t-1)}$, train locally, upload $\Delta W_k^{(t)}$, and the server updates $W^{(t)} = W^{(t-1)} + \text{Agg}(\{\Delta W_k^{(t)}\})$. The global objective is

$$\min_W \mathcal{L}(W) = \sum_{k=1}^K p_k \mathcal{L}_k(W), \quad p_k = |\mathcal{D}_k| / \sum_j |\mathcal{D}_j|. \quad (1)$$

The edge setting follows a client-edge-cloud workflow. The client side provides local training and an attestation-bound trust report, while the server audits updates, evolves trust states, and performs robust trust-weighted aggregation. The term secure aggregation is not used here: the server must inspect per-client evidence, so cryptographic privacy-preserving aggregation is outside the scope of this paper.

Threat model. The server is honest-but-curious and follows the aggregation protocol; it sees individual updates and audit statistics but is not allowed to alter the stated state-transition rules. An admitted client may control local labels, samples, or optimization objectives and may perform label flipping, trigger backdoors, clean-label poisoning, semantic backdoors, sign flipping, or gradient scaling. The trust boundary is explicit: the Intel and AMD TEEs protect attestation-critical code, device keys, report integrity, report freshness, and the trusted monitoring path against a compromised host OS, but they do not certify data labels, optimization objectives, or semantic intent. Thus, a malicious client may produce a signed poisoned update inside an otherwise valid measured training workflow; only post-attestation modification outside that workflow is excluded. If the report and update-binding check fail, the update is rejected rather than treated as a successful TEE defense. Physical probing and advanced microarchitectural side channels are also excluded. The analytical assumption is fewer than 50% malicious admitted clients; the 50% experiment is an out-of-assumption stress test, not a Byzantine threshold or a theoretical guarantee. The server-held clean proxy set is an explicit optional reference assumption, and

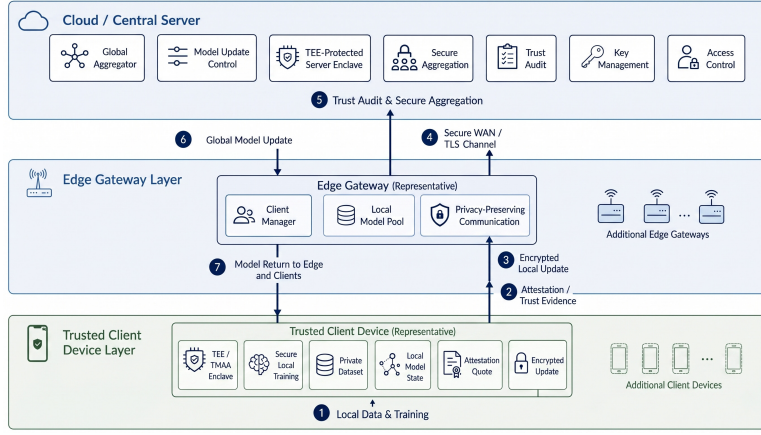


Fig. 1. TEE-anchored client-edge-cloud architecture.

the peer-reference fallback is defined for deployments that cannot provide such a set; no headline result is used to claim reference-free performance.

Design goals. Trust Flow aims to maintain convergence under the stated attacks, limit permanent false bans of long-tail clients, and measure server-side trust-flow cost separately from attestation and cryptographic privacy. It treats the recall-false-positive conflict as state separation: admission, content, history, and risk remain distinct until layer-wise aggregation.

4 Trust-Flow Framework

Trust Flow divides each FL round into four linked stages: admission and runtime trust assessment, reference-based content auditing, dual-stream state evolution, and risk-gated layered aggregation. Figure 3 shows the full defense loop.

The four stages output *TrustScore*, *ContentScore*, the slow *HistPerf*/fast *RiskEMA* states, and finally *RawScore*. Invalid proofs are rejected first; current updates are then audited, state is updated, and layer privileges are assigned. Keeping a low one-round content score provisional protects Non-IID clients, whereas persistent risk still causes isolation.

4.1 Admission and Content Auditing

Each admitted client must pass a TEE admission check. Let $M_{\text{attest},k} \in \{0,1\}$ denote whether the attestation report, its freshness nonce, and its binding to the round and update digest are valid. During initialization, the client invokes a device-unique key and generates a remote attestation quote containing a vendor-specific measurement digest, code identity, runtime configuration, round nonce, update digest, and monotonic freshness counter. Only clients passing this check receive communication privileges. During local training,

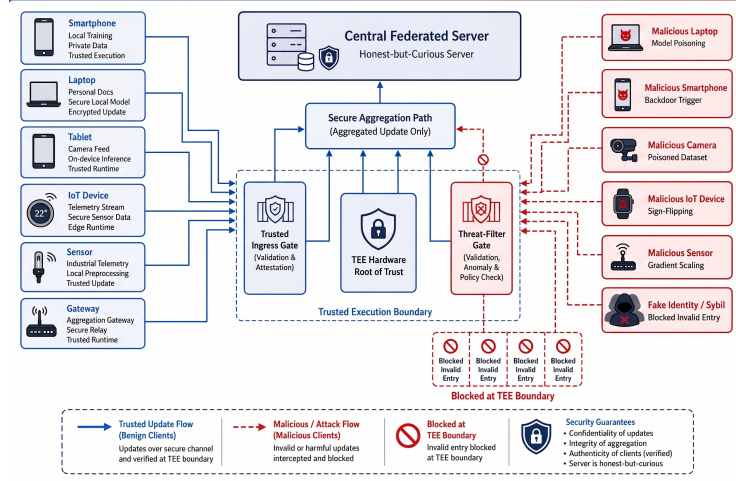


Fig. 2. Composite poisoning and penetration threat model.

the trusted management agent monitors standardized behavioral sequences $\mathcal{X}_k = \{\Delta\text{grad}, \Delta\text{loss}, \text{instr_dist}\}$. Let $h_k^{(t)} = H(\mathcal{X}_k^{(t)})/H_{\max} \in [0, 1]$ be the normalized entropy after concatenating the three standardized feature histograms, where $H_{\max}$ is the entropy of their uniform concatenation. We use

$$R_k^{(t)} = \exp(\lambda h_k^{(t)}), \quad Z_k^{(t)} = \text{clip}\left(w_a M_{\text{attest},k} + (1 - w_a)(1 - h_k^{(t)}), 0, 1\right), \quad (2)$$

where $w_a = 0.7$ and $\lambda = 1.0$. High entropy is therefore treated as uncertain evidence, not as a direct maliciousness label.

Following a scalar Kalman filtering view, the trust estimate is updated as

$$T_k^{-(t)} = T_k^{(t-1)}, \quad P_k^{-(t)} = P_k^{(t-1)} + Q, \quad (3)$$

$$\begin{aligned} K_k^{(t)} &= \frac{P_k^{-(t)}}{P_k^{-(t)} + R_k^{(t)}}, \\ T_k^{(t)} &= T_k^{-(t)} + K_k^{(t)}(Z_k^{(t)} - T_k^{-(t)}), \\ P_k^{(t)} &= (1 - K_k^{(t)})P_k^{-(t)}. \end{aligned} \quad (4)$$

The observation $Z_k^{(t)} \in [0, 1]$ is defined above; $Q = 0.01$ is the fixed process-noise variance and $R_k^{(t)}$ is the entropy-derived observation variance. We initialize a fresh admitted client with $T_k^{(0)} = 1$, $P_k^{(0)} = 0.25$, $\text{RiskEMA}_k^{(0)} = 0$, and $M_{\text{attest},k} = 1$. The resulting privilege factor is

$$\text{TrustScore}_k^{(t)} = M_{\text{attest},k} \text{clip}\left(\frac{T_k^{(t)}}{1 + P_k^{(t)}}, 0, 1\right). \quad (5)$$

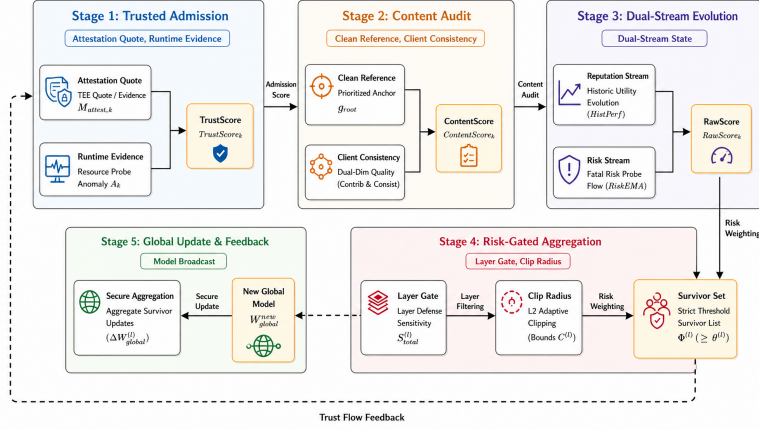


Fig. 3. Overview of the four-stage trust-flow defense loop.

High entropy increases $R_k^{(t)}$ and reduces the Kalman gain, so an uncertain observation cannot abruptly rewrite cross-round trust.

The server then constructs a clean guiding direction. If a small clean validation gradient is available, $g_{root} = g_{root_clean}$; otherwise, it uses admitted clients with high trust and low previous risk, with reference weights $\omega_k^{ref} = TrustScore_k^{(t)}(1 - RiskEMA_k^{(t-1)})^2$. In the reported experiments, a balanced 500-sample proxy set (50 images per CIFAR-10 class) is held by the server and is not included in client training. The same proxy set is supplied to every reference-dependent baseline and is also used for clean-loss and activation probes. When no such set is available, the trusted-peer fallback is used as a defined protocol branch; the headline table does not claim a measured reference-free advantage. For each update, content quality combines reference consistency and peer consistency:

$$S_{contrib,k} = \max(0, \alpha_1 + \alpha_2 \cos(\Delta W_k, g_{root})), \quad (6)$$

$$S_{consist,k} = \frac{\sum_{j \neq k} TrustScore_j \max(0, \alpha_1 + \alpha_2 \cos(\Delta W_k, \Delta W_j))}{\sum_{j \neq k} TrustScore_j + \varepsilon}, \quad (7)$$

$$ContentScore_k = \frac{(1 + \beta_{fusion}^2) S_{consist,k} S_{contrib,k}}{\beta_{fusion}^2 S_{consist,k} + S_{contrib,k} + \varepsilon}. \quad (8)$$

With $\beta_{fusion} > 1$, the harmonic fusion favors the clean direction but still requires peer plausibility, preventing either collusion or Non-IID reference deviation from dominating alone.

4.2 Dual-Stream State Evolution

Single reputation scores confuse low utility caused by data skew with high risk caused by attacks. Trust Flow therefore separates client state

into a historical utility stream and a short-term risk stream. Let $q_k^{(t)} = \text{clip}(\text{ContentScore}_k^{(t)}, 0, 1)$. The historical stream uses an explicitly decayed Beta model:

$$a_k^{(t)} = \beta_h a_k^{(t-1)} + q_k^{(t)}, \quad b_k^{(t)} = \beta_h b_k^{(t-1)} + 1 - q_k^{(t)}, \quad \text{HistPerf}_k^{(t)} = \frac{a_k^{(t)}}{a_k^{(t)} + b_k^{(t)} + \varepsilon}. \quad (9)$$

For a fresh client, $a_k^{(0)} = b_k^{(0)} = 1$, hence $\text{HistPerf}_k^{(0)} = 0.5$ rather than an unjustified perfect reputation. The decay factor is $\beta_h \in (0, 1)$ and is fixed before test rounds.

The history channel is intentionally slow, so low utility reduces privilege without immediate removal. The risk channel is fast: $\text{RiskEMA}_k^{(t)} = \beta_r \text{RiskEMA}_k^{(t-1)} + (1 - \beta_r) \text{Risk}_k^{\text{inst}}$, where $\text{Risk}_k^{\text{inst}}$ is the maximum current probe. Thus benign history cannot average away a strong warning.

A fresh client starts in NORMAL after valid attestation and cannot be permanently banned in round one. Its first-round content and risk values are therefore provisional: they can reduce the current update's privilege, but they cannot trigger an irreversible blacklist. Let $u_k^{(t)} = 1 - \text{HistPerf}_k^{(t)}$ and $r_k^{(t)} = \text{RiskEMA}_k^{(t)}$. The reported thresholds $(\tau_h, \tau_s, \tau_q, \tau_b) = (0.26, 0.64, 0.80, 0.90)$ govern utility deficit, soft risk, quarantine, and blacklisting. A NORMAL client enters SUSPECT after two consecutive rounds with $u_k^{(t)} > \tau_h$ or $r_k^{(t)} \geq \tau_s$. A SUSPECT client enters QUARANTINE after three such consecutive rounds or one current-round probe with $\text{Risk}_k^{\text{inst}} \geq \tau_q$. A QUARANTINE client enters BLACKLIST after four consecutive rounds with $r_k^{(t)} \geq \tau_b$. Recovery from SUSPECT or QUARANTINE requires two consecutive rounds with $u_k^{(t)} \leq \tau_h$ and $r_k^{(t)} < \tau_s$; a BLACKLIST state is irreversible within an experiment. These counters are reset when the corresponding condition is not met, and the state update is completed before layer aggregation.

The two streams meet only when computing aggregation privilege:

$$\begin{aligned} \text{RawScore}_k^{(t)} &= (\text{TrustScore}_k^{(t)})^\alpha (\text{ContentScore}_k^{(t)})^\beta (\text{HistPerf}_k^{(t)})^\gamma \\ &\quad \times (1 - \text{RiskEMA}_k^{(t)})^p. \end{aligned} \quad (10)$$

The state transition is evaluated after current probes and before aggregation. This timing lets current risk veto a sleeper burst despite high history, while a low-similarity benign client remains recoverable through the slow stream.

4.3 Risk-Gated Layered Aggregation

Backdoors often concentrate in deeper layers, whereas shallow feature layers may carry useful heterogeneous information. Trust Flow therefore assigns each layer l a sensitivity score. With L total layers, the three normalized components are

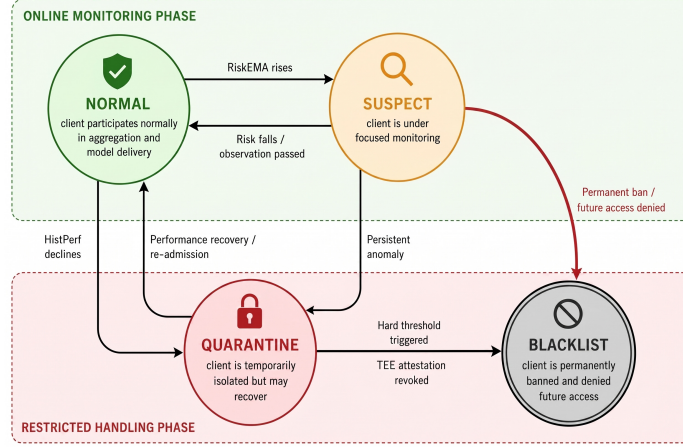


Fig. 4. Client state-transition automaton driven by historical utility and short-term risk.

$$\begin{aligned}
 S_{priv}^{(l)} &= \exp(-\tau_p l / L), \quad S_{util}^{(l)} = \frac{\|g_{root}^{(l)}\|_2}{\max_m \|g_{root}^{(m)}\|_2 + \varepsilon}, \\
 S_{sec}^{(l)} &= \frac{1}{2} \left(1 - \frac{1}{|\mathcal{T}|} \sum_{k \in \mathcal{T}} \cos(\Delta W_k^{(l)}, g_{root}^{(l)}) \right).
 \end{aligned} \tag{11}$$

where $\mathcal{T} = \{k \in \mathcal{A} : M_{attest,k} = 1, RiskEMA_k^{(t-1)} < \tau_s\}$ is the trusted reference set. If this set is empty, the server uses the admitted set $\mathcal{A}$ only to normalize the layer score and records that no trusted peer reference was available. Hence $S_{priv}^{(l)}, 1 - S_{util}^{(l)}, S_{sec}^{(l)} \in [0, 1]$. We use $S_{total}^{(l)} = w_p S_{priv}^{(l)} + w_u (1 - S_{util}^{(l)}) + w_s S_{sec}^{(l)}$, with $w_p, w_u, w_s = (0.3, 0.3, 0.4)$ and $w_p + w_u + w_s = 1$, so $S_{total}^{(l)} \in [0, 1]$. The threshold and clipping radius are

$$\theta^{(l)} = \mu_{base} + \lambda_s S_{total}^{(l)}, \quad C^{(l)} = \frac{C_{base}}{S_{total}^{(l)} + \varepsilon_c}. \tag{12}$$

Thus a layer with stronger suspicious drift becomes harder to enter and more tightly clipped; a layer that remains close to the reference keeps more capacity for benign heterogeneity.

We set $\tau_p = 3.0$, $\varepsilon = \varepsilon_c = 10^{-8}$, and define C_{base} as the median trusted-client update norm before layer gating. These choices keep the sensitivity terms dimensionless and make the clipping scale robust to the round-wise update magnitude.

Survivors are $\Phi^{(l)} = \{k \in \mathcal{A} \mid RawScore_k^{(t)} \geq \theta^{(l)}\}$, and their weights are re-normalized:

$$\tilde{w}_k^{(l)} = \frac{p_k RawScore_k^{(t)}}{\sum_{j \in \Phi^{(l)}} p_j RawScore_j^{(t)} + \varepsilon}, \quad \widehat{\Delta W}_k^{(l)} = \frac{\Delta W_k^{(l)}}{\max(1, \|\Delta W_k^{(l)}\|_2 / C^{(l)})}. \tag{13}$$

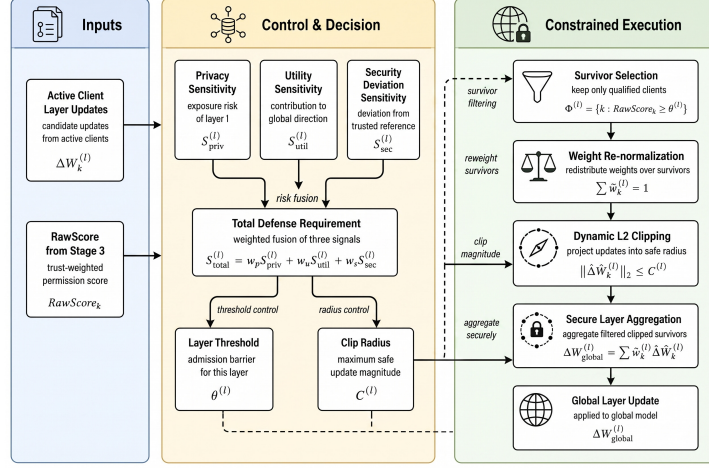


Fig. 5. Risk-gated layer-wise clipping and differentiated aggregation.

The final update is $\Delta W_{global}^{(l)} = \sum_{k \in \Phi^{(l)}} \tilde{w}_k^{(l)} \widehat{\Delta W}_k^{(l)}$. Weight re-normalization prevents the removal of risky clients from shrinking the effective learning rate.

If $\Phi^{(l)} = \emptyset$ because a highly sensitive layer rejects all candidates, the server retains $k^* = \arg \max_{k \in \mathcal{A}} RawScore_k^{(t)}$ for that layer, assigns it unit normalized weight, and still applies the clipping radius $C^{(l)}$. This fallback keeps the layer update defined without weakening the threshold for the other clients.

This layer-wise control preserves useful shallow representations while hardening layers with stronger suspicious drift. A semantic backdoor that mainly shifts deep features can therefore be clipped or rejected in those layers without imposing the same threshold on benign shallow features. Re-normalization is coupled to this decision: after unsafe mass is removed, the surviving data-size-weighted scores are projected back to a valid aggregation simplex, avoiding an implicit learning-rate decay.

4.4 Risk Probe Computation

The short-term risk stream uses probes that are intentionally redundant rather than individually decisive. For update k in round t , the normalized channels are

$$r_{grad,k}^{(t)} = \text{clip}\left(\frac{1 - \cos(\Delta W_k, g_{root})}{2}, 0, 1\right), \quad (14)$$

$$r_{amp,k}^{(t)} = \text{clip}\left(\frac{\|\Delta W_k\|_2}{\text{median}_j \|\Delta W_j\|_2 + \varepsilon} - 1, 0, 1\right), \quad (15)$$

$$r_{clean,k}^{(t)} = \text{clip}\left(\frac{\mathcal{L}_{probe}(W + \Delta W_k) - \mathcal{L}_{probe}(W)}{\delta_{clean} + \varepsilon}, 0, 1\right), \quad (16)$$

$$r_{trigger,k}^{(t)} = \text{clip}\left(\frac{\text{KL}(\mathcal{H}(A_{base})\|\mathcal{H}(A_k))}{\delta_{trigger} + \varepsilon}, 0, 1\right), \quad (17)$$

$$\bar{c}_k^{(t)} = \frac{\sum_{j \neq k} \text{TrustScore}_j c_{kj}}{\sum_{j \neq k} \text{TrustScore}_j + \varepsilon}, \quad r_{peer,k}^{(t)} = \text{clip}(1 - \bar{c}_k^{(t)}, 0, 1), \quad (18)$$

where $c_{kj} = \cos(\Delta W_k, \Delta W_j)$. For the trigger probe, A_{base} is the activation histogram of the current global model on the server-held proxy set and A_k is the corresponding histogram after applying candidate update k . We use 32-bin histograms from the last convolutional block and the pre-classifier representation. In each round, δ_{clean} and $\delta_{trigger}$ are set to the median plus three median absolute deviations of the corresponding values over $\mathcal{T}$, making the calibration independent of the candidate under test. Define $Risk_k^{inst} = \max\{1 - M_{attest,k}, r_{grad,k}, r_{amp,k}, r_{clean,k}, r_{trigger,k}, r_{peer,k}\}$.

The maximum preserves complementary warnings: gradient and amplitude channels expose parameter attacks, whereas clean-loss and activation channels target stealthier backdoors. Peer disagreement is supporting evidence because benign long-tail updates may disagree with the majority. The execution order is reference construction, current-update audit, state update, and layer aggregation.

5 Experiments

5.1 Setup

Experiments use Flower 1.5.0 and PyTorch on CIFAR-10 with a CIFAR-10-adapted ResNet-18: the first convolution uses a 3×3 kernel with stride one, the initial max-pooling layer is removed, and the classifier has ten outputs. Training and trusted execution are performed in two real hardware configurations based on Intel and AMD TEE-capable processors. NVIDIA A10 GPUs execute model training, whereas the CPU-side TEEs isolate attestation, trusted monitoring, report generation, and admission-state processing. Both hardware configurations execute the same request/response procedure, nonce-based freshness verification, measured-code identity checking, round and update-digest binding, protected report generation, and admission decision. The testbed uses anonymized servers

with Intel and AMD dual-CPU configurations and five NVIDIA A10 GPUs; the software stack is Ubuntu 20.04.6 LTS and CUDA 12.8. The 50,000 training images are partitioned into a shared 5,000-image client pool, replicated as a common base subset for all clients, and 45,000 exclusive images: clients 0–9 use an IID split, clients 10–14 use Dirichlet $\alpha = 1.0$, and clients 15–19 use Dirichlet $\alpha = 0.1$. Separately, the server holds a balanced 500-image proxy set (50 per class) from CIFAR-10 test data; it is not used in client training and is supplied to every reference-dependent baseline for the guiding gradient and probes. FedAvg, Krum, and Trimmed Mean do not consume the proxy set. All methods use the same optimizer and communication budget. Five independent seeds are run on each hardware configuration; means and standard deviations are computed over the resulting ten platform-seed runs.

The formal 30% attack setting uses six malicious clients: Client 0 performs untargeted label flipping with $\gamma = 0.5$; Client 1 injects a fixed 3×3 lower-right trigger into 20% of local samples and targets class 0; Client 2 uses label-preserving clean-label perturbations on 20% of local samples with the same target; Client 3 applies a fixed semantic attribute bias to 20% of local samples, with the biased subset assigned to class 0; Client 4 applies untargeted sign flipping; and Client 5 applies untargeted gradient scaling by a factor of 100. Clients 1–3 are evaluated as targeted attacks, whereas Clients 0, 4, and 5 are evaluated through clean loss, convergence, and detection state. Attack behaviors are active in every local training round; no attack is introduced only after observing the global trajectory. Five Sybil nodes without valid certificates and three free-rider nodes with forged low workload are used only to validate Phase-One gating. These eight nodes are intercepted before statistical aggregation, so later metrics are computed over the 20 admitted clients. Malicious updates that pass hardware attestation represent an attacker operating inside an admissible training process; post-attestation host tampering is excluded by the stated threat model. Each run uses 30 rounds and three local epochs. Aggregate values are averaged over the pooled platform-seed runs; state trajectories use one fixed seed per platform.

Clean accuracy and targeted ASR are reported as mean and standard deviation over the ten platform-seed runs. For each targeted behavior $a \in \{\text{trigger}, \text{clean}, \text{semantic}\}$, let $\mathcal{B}_a$ be its trigger- or attribute-bearing test subset and y_a^* its declared target. We define $ASR_a = |\{x \in \mathcal{B}_a : \arg \max f(x) = y_a^*\}|/|\mathcal{B}_a|$ and report the macro-average $ASR_{\text{target}} = \frac{1}{3} \sum_a ASR_a$. The three attack-specific quantities are retained separately in the convergence traces; the headline table uses the macro-average to avoid assigning a larger weight to any one targeted behavior. Untargeted label/sign/scaling behavior is not folded into this numerator and is assessed through clean accuracy, loss, and state-transition time. Permanent-ban FPR is computed only for benign clients that are irreversibly blacklisted, while temporary SUSPECT or QUARANTINE states are recoverable control actions. Stateless baselines have no equivalent permanent-ban state, so their FPR is reported as N/A rather than compared numerically. All baselines are executed under the same node partitioning, attack

Table 1. Experimental configuration and main hyperparameters.

Category	Setting	Value
FL task	Dataset/model/admitted clients	CIFAR-10/CIFAR-adapted ResNet-18/ $K = 20$
Optimization	Local epochs, lr, momentum, rounds	3, 0.001, 0.9, 30
Data heterogeneity	Shared client pool; server proxy; Group A/B/C	5000; 500; $\alpha = 1.0, 0.1$
Trust streams	$\alpha_1, \alpha_2, \beta_{fusion}; \beta_h, \beta_r$	0.6, 0.4, 2.0; 0.9, 0.85
Admission filter	$w_a, \lambda, Q; T_0, P_0, RiskEMA_0$	0.7, 1.0, 0.01; 1, 0.25, 0
Layer gating	$\alpha, \beta, \gamma, p; \mu_{base}, \lambda_s$	3.0, 1.0, 0.5, 1.0; 0.4, 1.2
Sensitivity	$\tau_p; w_p, w_u, w_s$	3.0; 0.3, 0.3, 0.4
Numerical stability	$\varepsilon, \varepsilon_c; C_{base}$	$10^{-8}, 10^{-8}$; median trusted norm
State machine	$\tau_h, \tau_s, \tau_q, \tau_b$; persistence	0.26, 0.64, 0.80, 0.90; 2, 3, 4 rounds

proportion, optimizer configuration, communication rounds, reference access, and Non-IID split.

5.2 Overall Defense Performance

Figure 6 summarizes convergence, the three attack-specific ASR traces, trust-state trajectories, and feature separation. Under the stated 30% mixed-attack protocol, all six attack behaviors are active concurrently in every local round; this is a simultaneous-attack test rather than a collection of one-at-a-time trials. Trust Flow reaches 92.31% clean accuracy and 10.21% macro-averaged targeted ASR, close to the 10% random-target rate for ten CIFAR-10 classes. Client-state tracing shows that explicit poisoning and parameter attacks are blocked early, while stealthier clean-label drift is captured after persistent probe evidence. Legitimate long-tail nodes may enter SUSPECT or QUARANTINE temporarily, but their risk stream does not accumulate enough evidence for permanent banning. These observations describe the tested protocol and do not establish universal superiority over recent defenses that use different probes, reference data, or state definitions.

Trajectories show sustained risk for label, trigger, semantic, sign, and scaling attacks, whereas a long-tail benign client recovers because its risk does not persist. The t-SNE view similarly separates high-risk clients from benign heterogeneous updates after dual-stream supervision.

Within the matched classical comparison, Krum reaches 64.20% accuracy, Trimmed Mean 71.35%, and FLTrust 81.25%, compared with 92.31% for Trust Flow. These values are not a universal ranking: recent defenses such as DeepSight, FLAME, CrowdGuard, RoseAgg, FLPurifier, ShieldFL, and RaSA require attack-specific or privacy-preserving interfaces that are not reproduced in this submission. Their absence is therefore treated as a comparison limitation rather

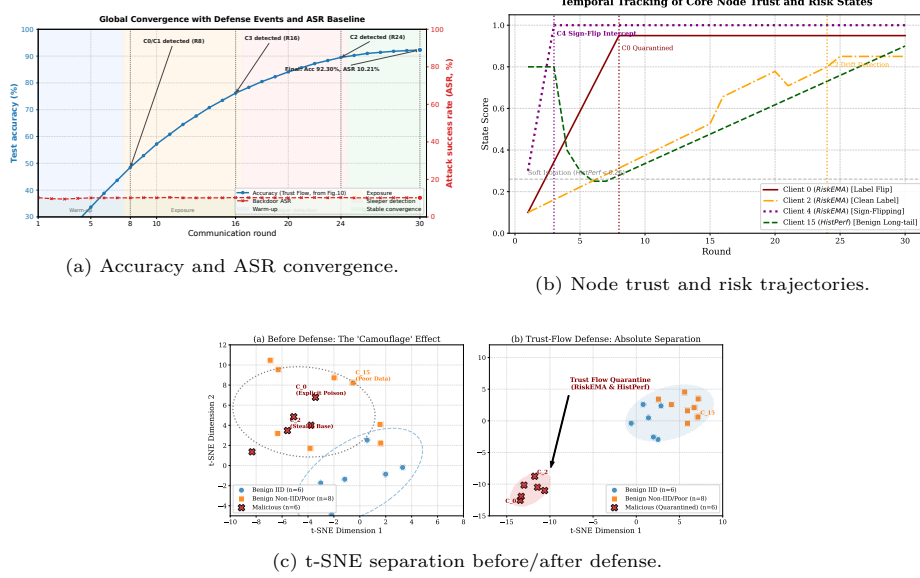


Fig. 6. Main defense behavior under mixed attacks.

than as evidence that Trust Flow dominates those methods. The 50% result is an out-of-assumption stress test, not evidence of a theoretical Byzantine boundary.

5.3 Ablation, Sensitivity, and Overhead

Table 3 exposes the single-stream FPR/TPR tradeoff and the low recall of HistPerf-only. Measurements from the Intel and AMD TEE deployments show that the attestation-bound TrustReport adds about 15 KB and that aggregation plus audit increases from 2.10 s to 4.85 s. Optional cryptographic secure aggregation remains outside the stated system boundary.

Figures 7 and 8 show that counting SUSPECT as recoverable review yields 88.33% per-round malicious coverage and a 14.79% benign review rate, with no permanent false ban in the tested runs. Activation and clean-loss probes reduce the reported clean-label ASR below 9.2%, while re-normalization avoids implicit learning-rate decay. At $\alpha = 0.1$, Trust Flow remains near 92% accuracy on the tested hardware platforms. These ablation values are descriptive measurements under the same protocol; they are not claims of statistical significance beyond the reported repeated-run summary.

The Dirichlet sweep in Fig. 8 is a within-family distribution-shift test: the label space, model, optimizer, and attack protocol remain fixed while the concentration parameter changes from weak to strong heterogeneity. As α decreases, Krum and FLTrust increasingly reject legitimate drift; Trust Flow is less sensitive in the reported sweep because content is fused with history and risk. The default $\beta_{fusion} = 2.0$ gives the best observed accuracy-ASR balance. The

Table 2. Performance comparison under 30% mixed attacks and boundary stress testing.

Method/Setting	Core mechanism	Acc.	Targeted ASR	Perm. FPR [†]
FedAvg [2]	no defense	86.15 ± 1.20	92.34 ± 5.10	N/A
Krum [3]	Euclidean selection	64.20 ± 2.80	12.15 ± 0.90	N/A
Trimmed Mean [4]	coordinate trimming	71.35 ± 2.40	38.60 ± 4.20	N/A
FLTrust [5]	clean anchor weighting	81.25 ± 1.50	15.42 ± 1.10	N/A
Trust Flow	dual stream + layered gating	92.31 ± 0.09	10.21 ± 0.05	0.00
Boundary stress: 50% malicious	10/20 malicious	91.81 ± 0.10	10.46 ± 0.06	0.00

[†] Permanent-ban FPR is defined only for Trust Flow and its stateful ablations. Krum, Trimmed Mean, and FLTrust are stateless in this comparison and are therefore marked N/A. Temporary SUSPECT/QUARANTINE states are recoverable and reported separately through trajectories.

Table 3. Ablation and overhead summary.

Study	Variant / architecture	FPR or overhead	TPR / time
Reputation	Single-stream EMA, aggressive	18.25% FPR	95.12% TPR
Reputation	Single-stream EMA, conservative	2.10% FPR	48.33% TPR
Reputation	HistPerf-only	0.15% FPR	21.05% TPR
Reputation	Dual-stream Trust Flow	0.00% FPR	100.00% TPR
Overhead	Standard FedAvg	0 KB extra	2.10 s aggregation
Overhead	Trust Flow	~15 KB TrustReport	4.85 s incl. audit

reported runtime covers server-side aggregation and auditing only; it excludes client communication, enclave transitions, memory pressure, and vendor-specific attestation latency, so it is a protocol cost estimate rather than an end-to-end deployment measurement.

6 Discussion and Conclusion

Trust Flow integrates admission evidence, content auditing, temporal reputation, and layer-wise aggregation into a single trust-state pipeline. The system-level contribution is the ordering and separation of these evidence streams: execution evidence controls admission, current probes control immediate risk, historical utility controls recoverable retention, and layer scores control where an update can contribute. On the two tested Intel and AMD TEE hardware configurations, this separation keeps permanent-ban FPR near zero while maintaining high recall against the tested mixed attacks. The deployments verify the stated attestation interface, evidence binding, protected reporting, and transition semantics; TEE admission nevertheless remains distinct from semantic update-safety verification.

Several limitations remain. Microarchitectural side channels and vendor-specific enclave resource contention remain outside the threat model. The clean-reference and activation-probe assumptions require a declared server-held probe set and may not transfer directly to strict cross-silo or cross-device

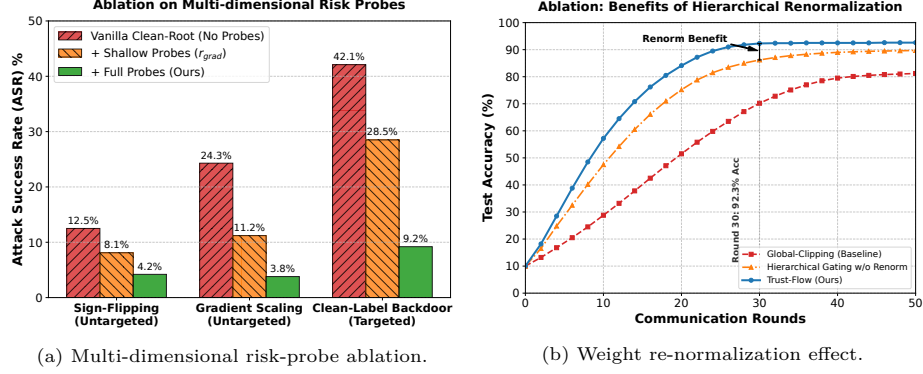


Fig. 7. Component-level ablation of risk probes and re-normalization.

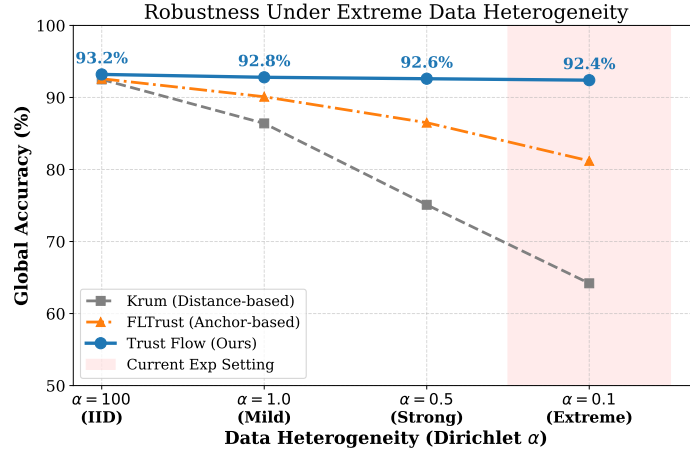


Fig. 8. Robustness stress test under varying data heterogeneity.

settings without trusted domain data; the peer-reference branch is defined but not evaluated in the headline results. The comparison includes matched classical baselines but does not provide direct reimplementations of every recent temporal, backdoor, or privacy-preserving defense; consequently, the numerical results should not be read as a universal ranking. The current probe design is also tailored to vision classification, and cryptographic secure aggregation remains outside the system boundary because the server must inspect individual updates. These limitations motivate reference-free evaluation, matched modern baselines, and broader task validation in future work.

References

1. Kairouz, P., McMahan, H.B., Avent, B., et al.: Advances and open problems in federated learning. *Foundations and Trends in Machine Learning* 14(1–2), 1–210 (2021). <https://doi.org/10.1561/22000000083>
2. McMahan, B., Moore, E., Ramage, D., Hampson, S., Aguera y Arcas, B.: Communication-efficient learning of deep networks from decentralized data. In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, PMLR 54, pp. 1273–1282 (2017). <https://proceedings.mlr.press/v54/mcmahan17a.html>
3. Blanchard, P., El Mhamdi, E.M., Guerraoui, R., Stainer, J.: Machine learning with adversaries: Byzantine tolerant gradient descent. In: *Advances in Neural Information Processing Systems* 30 (2017). [NeurIPS paper page](#)
4. Yin, D., Chen, Y., Ramchandran, K., Bartlett, P.: Byzantine-robust distributed learning: Towards optimal statistical rates. In: *Proceedings of the 35th International Conference on Machine Learning*, PMLR 80, pp. 5650–5659 (2018). <https://proceedings.mlr.press/v80/yin18a.html>
5. Cao, X., Fang, M., Liu, J., Gong, N.Z.: FLTrust: Byzantine-robust federated learning via trust bootstrapping. In: *Proceedings 2021 Network and Distributed System Security Symposium (NDSS)* (2021). <https://doi.org/10.14722/ndss.2021.24434>
6. Fung, C., Yoon, C.J.M., Beschastnikh, I.: The limitations of federated learning in Sybil settings. In: *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, pp. 301–316. USENIX Association (2020). <https://www.usenix.org/conference/raid2020/presentation/fung>
7. Wang, L., Huang, M., Zhang, Z., Li, M., Wang, J., Gai, K.: RaSA: Robust and adaptive secure aggregation for edge-assisted hierarchical federated learning. *IEEE Transactions on Information Forensics and Security* 20, 4280–4295 (2025). <https://doi.org/10.1109/TIFS.2025.3559411>
8. Zhang, Z., Cao, X., Jia, J., Gong, N.Z.: FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients. In: *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 2545–2555 (2022). <https://doi.org/10.1145/3534678.3539231>
9. Rieger, P., Nguyen, T.D., Miettinen, M., Sadeghi, A.R.: DeepSight: Mitigating backdoor attacks in federated learning through deep model inspection. In: *Network and Distributed System Security Symposium (NDSS)* (2022). <https://doi.org/10.14722/ndss.2022.23156>
10. Nguyen, T.D., Rieger, P., Chen, H., et al.: FLAME: Taming backdoors in federated learning. In: *31st USENIX Security Symposium*, pp. 1415–1432 (2022). <https://www.usenix.org/conference/usenixsecurity22/presentation/nguyen>
11. Zhang, J., Zhu, C., Sun, X., Ge, C., Chen, B., Susilo, W., Yu, S.: FLPurifier: Backdoor defense in federated learning via decoupled contrastive training. *IEEE Transactions on Information Forensics and Security* 19, 4752–4766 (2024). <https://doi.org/10.1109/TIFS.2024.3384846>
12. Rieger, P., Krauß, T., Miettinen, M., Dmitrienko, A., Sadeghi, A.R.: CrowdGuard: Federated backdoor detection in federated learning. In: *Network and Distributed System Security Symposium (NDSS)* (2024). <https://doi.org/10.14722/ndss.2024.23233>
13. Yang, H., Xi, W., Shen, Y., Wu, C., Zhao, J.: RoseAgg: Robust defense against targeted collusion attacks in federated learning. *IEEE Transactions on Information*

- Forensics and Security 19, 2951–2966 (2024). <https://doi.org/10.1109/TIFS.2024.3352415>
14. Ma, Z., Ma, J., Miao, Y., Li, Y., Deng, R.H.: ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning. *IEEE Transactions on Information Forensics and Security* 17, 1639–1654 (2022). <https://doi.org/10.1109/TIFS.2022.3169918>
 15. Chen, Y., Luo, F., Li, T., Xiang, T., Liu, Z., Li, J.: A training-integrity privacy-preserving federated learning scheme with trusted execution environment. *Information Sciences* 522, 69–79 (2020). <https://doi.org/10.1016/j.ins.2020.02.037>
 16. Xu, T., Zhu, K., Andrzejak, A., Zhang, L.: Distributed learning in trusted execution environment: A case study of federated learning in SGX. In: 2021 7th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC), pp. 450–454 (2021). <https://doi.org/10.1109/IC-NIDC54101.2021.9660433>
 17. Zhang, X., Chen, Z., Chen, G., Feng, X., Shen, Q., Wu, Z.: RPPFL: Robust and privacy-preserving federated learning via trusted execution environments. In: ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing, pp. 1–5 (2025). <https://doi.org/10.1109/ICASSP49660.2025.10889398>
 18. Queyrut, S., Schiavoni, V., Felber, P.: Mitigating adversarial attacks in federated learning with trusted execution environments. In: 2023 IEEE 43rd International Conference on Distributed Computing Systems (ICDCS), pp. 626–637 (2023). <https://doi.org/10.1109/ICDCS57875.2023.00069>
 19. Lu, Z., Wang, L., Zhang, Z., Huang, M., Wang, J., Li, M.: TMT-FL: Enabling trustworthy model training of federated learning with malicious participants. *IEEE Transactions on Dependable and Secure Computing* 22(3), 2723–2740 (2025). <https://doi.org/10.1109/TDSC.2024.3521377>
 20. Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., Shmatikov, V.: How To Backdoor Federated Learning. In: *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, PMLR 108, pp. 2938–2948 (2020). <https://proceedings.mlr.press/v108/bagdasaryan20a.html>
 21. Wang, B., Yao, Y., Shan, S., Li, H., Viswanath, B., Zheng, H., Zhao, B.Y.: Neural Cleanse: Identifying and mitigating backdoor attacks in neural networks. In: 2019 IEEE Symposium on Security and Privacy (SP), pp. 707–723 (2019). <https://doi.org/10.1109/SP.2019.00031>